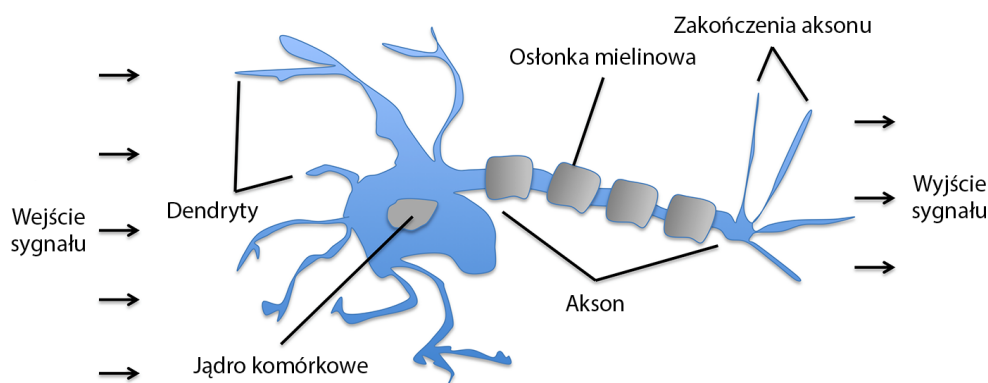


Sztuczne neurony — rys historyczny początków uczenia maszynowego

Zanim przejdziemy do dokładnego opisu modelu perceptronu oraz powiązanych z nim algorytmów, cofnijmy się na chwilę do początków dziedziny uczenia maszynowego. Warren McCulloch i Walter Pitts pragnęli zrozumieć mechanizm działania mózgu po to, aby zaprojektować sztuczną inteligencję, i w 1943 roku zaprezentowali pierwszą koncepcję uproszczonego modelu komórki nerwowej, tzw. **neuronu McCullocha-Pittsa** (ang. *McCulloch-Pitts neuron* — **MCP**; W.S. McCulloch i W. Pitts, *A Logical Calculus of the Ideas Immanent in Nervous Activity*, „The Bulletin of Mathematical Biophysics” 1943, nr 5 (4), s. 115 – 133). Neuronami nazywamy wzajemnie połączone komórki nerwowe w mózgu, które są odpowiedzialne za przetwarzanie oraz przesyłanie sygnałów chemicznych i elektrycznych, co zostało zaprezentowane na rysunku 2.1.



Rysunek 2.1. Model uproszczonego neuronu

McCulloch i Pitts opisali taką komórkę nerwową jako prostą bramkę logiczną zawierającą binarne wyjścia; do dendrytów dociera wiele sygnałów, które są integrowane w ciele komórki i, jeżeli energia impulsu przekracza określoną wartość graniczną, zostaje wygenerowany sygnał wyjściowy przepuszczany poprzez akson.

Już kilka lat później Frank Rosenblatt na podstawie modelu neuronu MCP opublikował pierwszą koncepcję reguły uczenia perceptronu (F. Rosenblatt, *The Perceptron, a Perceiving and Recognizing Automaton*, „Cornell Aeronautical Laboratory”, 1957). Korzystając z tej reguły, Rosenblatt zaproponował algorytm zdolny do automatycznego uczenia się za pomocą optymalnych współczynników wag, które są przemnażane przez wartości wejściowe, co pozwala określić, czy neuron prześle dalej sygnał. W kontekście uczenia nadzorowanego i klasyfikacji taki algorytm może być wykorzystywany do przewidywania próbek przynależnych do różnych klas.

W ujęciu matematycznym możemy przedstawić ten problem jako klasyfikację binarną, w której dla uproszczenia odnosimy się do dwóch klas: 1 (klasy pozytywnej) oraz -1 (klasy negatywnej). Następnie definiujemy **funkcję aktywacji** $\phi(z)$, na którą składa się liniowa kombinacja określonych wartości wejściowych x oraz powiązanego z nimi wektora wag w , gdzie z nosi nazwę całkowitego pobudzenia układu ($z = w_1x_1 + \dots + w_mx_m$):

$$\mathbf{w} = \begin{bmatrix} w_1 \\ \vdots \\ w_m \end{bmatrix}, \mathbf{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_m \end{bmatrix}$$

Jeżeli całkowite pobudzenie z danej próbki $\mathbf{x}^{(i)}$ jest wyższe od zdefiniowanej wartości progowej θ , to przewidujemy, że dany obiekt przynależy do klasy pozytywnej 1, w przeciwnym wypadku — do klasy negatywnej -1 . W algorytmie perceptronu funkcja aktywacji $\varphi(\cdot)$ jest prostą funkcją skoku jednostkowego, zwaną czasem również funkcją skokową Heaviside'a:

$$\varphi(z) = \begin{cases} 1 & \text{jeśli } z \geq \theta \\ -1 & \text{jeśli } z < \theta \end{cases}$$

Możemy dla uproszczenia przenieść wartość progową θ na lewą stronę równania i zdefiniować początkową wagę jako $w_0 = -\theta$, a $x_0 = 1$, dzięki czemu całkowite pobudzenie z przybierze

prostsza postać $z = w_0x_0 + w_1x_1 + \dots + w_mx_m = \mathbf{w}^T \mathbf{x}$ przy założeniu, że $\varphi(z) = \begin{cases} 1 & \text{jeśli } z \geq 0 \\ -1 & \text{jeśli } z < 0 \end{cases}$.

W kolejnych podrozdziałach będziemy często stosować podstawową notację z zakresu algebry liniowej, np. korzystać ze skróconego zapisu sumy iloczynów wartości x i w za pomocą **iloczynu skalarnego wektorów**, gdzie indeks górny T oznacza **transpozycję** — operację przekształcania wiersza wektora w kolumnę i odwrotnie:

$$z = w_0x_0 + w_1x_1 + \dots + w_mx_m = \sum_{j=0}^m \mathbf{x}_j \mathbf{w}_j = \mathbf{w}^T \mathbf{x}$$

Na przykład: $\begin{bmatrix} 1 & 2 & 3 \end{bmatrix} \times \begin{bmatrix} 4 \\ 5 \\ 6 \end{bmatrix} = 1 \times 4 + 2 \times 5 + 3 \times 6 = 32.$

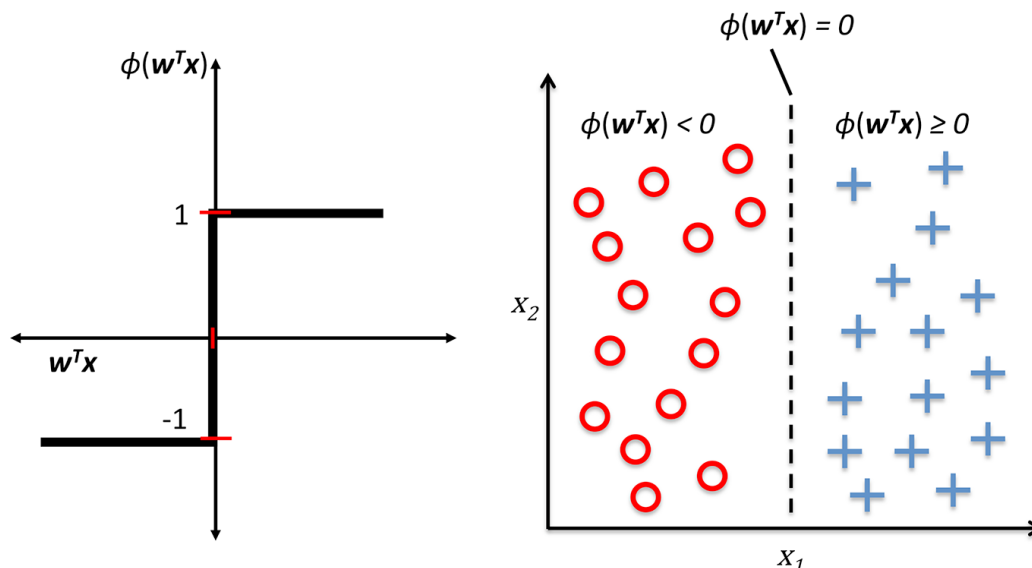
Ponadto operację transponowania można przeprowadzić również wobec macierzy, dzięki czemu następuje w niej zamiana wierszy z kolumnami:

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix}^T = \begin{bmatrix} 1 & 3 & 5 \\ 2 & 4 & 6 \end{bmatrix}$$

W tej książce będziemy wykorzystywać jedynie najprostsze pojęcia z algebry liniowej, jeżeli jednak chcesz odświeżyć sobie pamięć, polecam znakomity skrypt *Linear Algebra Review and Reference* autorstwa Zico Koltera, który można bezpłatnie przejrzeć na stronie http://www.cs.cmu.edu/~zkolter/course/linalg/linalg_notes.pdf¹.

¹ Dobrym odpowiednikiem w języku polskim jest skrypt *Matematyka dla studiów inżynierskich. Część 1. Algebra i geometria* autorstwa Stanisława Białasa, Adama Ćmiela i Andrzeja Fitzkego, dostępny pod adresem <http://winntbg.bg.agh.edu.pl/skrypty2/0077/bialas.pdf> — przyp. tłum.

Na rysunku 2.2 widzimy, w jaki sposób całkowite pobudzenie $z = \mathbf{w}^T \mathbf{x}$ zostaje przetworzone na wartości binarne (-1 lub 1) przez funkcję aktywacji perceptronu (wykres po lewej), a także jak może zostać wykorzystane do rozdzielenia dwóch odrębnych liniowo klas (wykres po prawej).



Rysunek 2.2. Zastosowanie całkowitego pobudzenia w uczeniu maszynowym

Podstawowym założeniem w neuronie MCP i modelu perceptronu **progowego** jest wprowadzenie uproszczonego mechanizmu naśladującego działanie pojedynczej komórki nerwowej: albo zostaje ona **uaktywniona**, albo nie. Z tego powodu pierwotna reguła uczenia perceptronu autorstwa Rosenblatta jest całkiem nieskomplikowana i można ją opisać następującymi etapami:

1. Wprowadź wagi o wartości 0 lub niewielkich, losowych wartościach.
2. Dla każdej próbki uczącej $\mathbf{x}^{(i)}$ wykonaj poniższe czynności:
 - a) Oblicz wartość wyjściową $\hat{y}$.
 - b) Zaktualizuj wagi.

W tym przypadku wartością wyjściową jest etykieta klasy przewidziana przez wcześniej zdefiniowaną funkcję skoku jednostkowego, a równoczesną aktualizację każdej wagi w_j w wektorze wag $\mathbf{w}$ można zapisać w bardziej formalny sposób:

$$w_j := w_j + \Delta w_j$$

Służąca do aktualizowania wagi w_j wartość Δw_j jest wyliczana za pomocą reguły uczenia perceptronu:

$$\Delta w_j = \eta (y^{(i)} - \hat{y}^{(i)}) x_j^{(i)}$$

gdzie η jest współczynnikiem uczenia (stałą przyjmującą wartości w zakresie od 0 do 1), $y^{(i)}$ stanowi rzeczywistą etykietę klas i -tej próbki uczącej, natomiast $\hat{y}^{(i)}$ to przewidywana etykieta

klas. Bardzo istotna jest informacja, że wszystkie wagi w wektorze wag są jednocześnie aktualizowane, co oznacza, że nie możemy ponownie przeliczyć wartości $\hat{y}^{(i)}$, dopóki nie zaktualizujemy wszystkich wag $\otimes w_j$.

Zapis aktualizacji dla dwuwymiarowego zbioru danych możemy zdefiniować następująco:

$$\begin{aligned}\Delta w_0 &= \eta(y^{(i)} - \text{wyjście}^{(i)}) \\ \Delta w_1 &= \eta(y^{(i)} - \text{wyjście}^{(i)})x_1^{(i)} \\ \Delta w_2 &= \eta(y^{(i)} - \text{wyjście}^{(i)})x_2^{(i)}\end{aligned}$$

Zanim zaimplementujemy model perceptronu w Pythonie, przeprowadźmy mały eksperyment myślowy ukazujący piękno prostoty tej reguły uczenia. W dwóch scenariuszach, w których perceptron we właściwy sposób przewiduje etykietę klas, wagi pozostają niezmienione:

$$\begin{aligned}\Delta w_j &= \eta(-1 - -1)x_j^{(i)} = 0 \\ \Delta w_j &= \eta(1 - 1)x_j^{(i)} = 0\end{aligned}$$

Jednak w przypadku nieprawidłowego prognozowania wagi zostają przesunięte w kierunku pozytywnej lub negatywnej klasy docelowej, odpowiednio:

$$\begin{aligned}\Delta w_j &= \eta(1 - -1)x_j^{(i)} = \eta(2)x_j^{(i)} \\ \Delta w_j &= \eta(-1 - 1)x_j^{(i)} = \eta(-2)x_j^{(i)}\end{aligned}$$

Aby lepiej zrozumieć koncepcję mnożnika $x_j^{(i)}$, przyjrzyjmy się kolejnemu prostemu przykładowi, w którym:

$$y^{(i)} = +1, \hat{y}^{(i)} = -1, \eta = 1$$

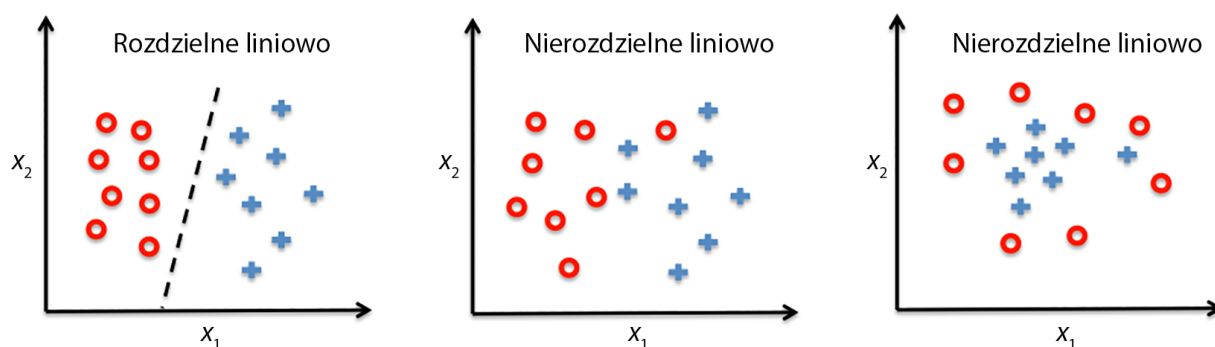
Założmy, że $x_j^{(i)} = 0,5$, a my tę próbkę nieprawidłowo sklasyfikowaliśmy jako -1 . W takim przypadku zwiększamy wagę o 1, przez co całkowite pobudzenie $x_j^{(i)} \times w_j$ będzie silniejsze w sytuacji ponownego natrafienia na tę próbkę, dzięki czemu z większym prawdopodobieństwem zostanie przekroczona wartość graniczna funkcji skokowej, a badany obiekt zostanie zaklasyfikowany do klasy $+1$:

$$\Delta w_j^{(i)} = (1 - -1)0,5 = (2)0,5 = 1$$

Aktualizacja wagi jest wprost proporcjonalna do wartości $x_j^{(i)}$. Założmy, że mamy kolejną próbkę, $x_j^{(i)} = 2$, która została nieprawidłowo zaklasyfikowana jako -1 . W tej sytuacji przesuwamy granicę decyzyjną w jeszcze większym stopniu po to, aby próbka została następnym razem właściwie zaklasyfikowana:

$$\Delta w_j = (1 - -1)2 = (2)2 = 4$$

Zwróć uwagę, że zbieżność perceptronu zostaje zapewniona jedynie wtedy, gdy dwie klasy są liniowo rozdzielne (rysunek 2.3), a współczynnik uczenia jest wystarczająco mały. Jeżeli nie można oddzielić dwóch klas za pomocą liniowej granicy decyzyjnej, możemy ustalić maksymalną liczbę przebiegów (**epok**) algorytmu z wykorzystaniem danych uczących i (lub) próg tolerancji nieprawidłowych klasyfikacji — w przeciwnym wypadku perceptron wiecznie aktualizowałby wagi.

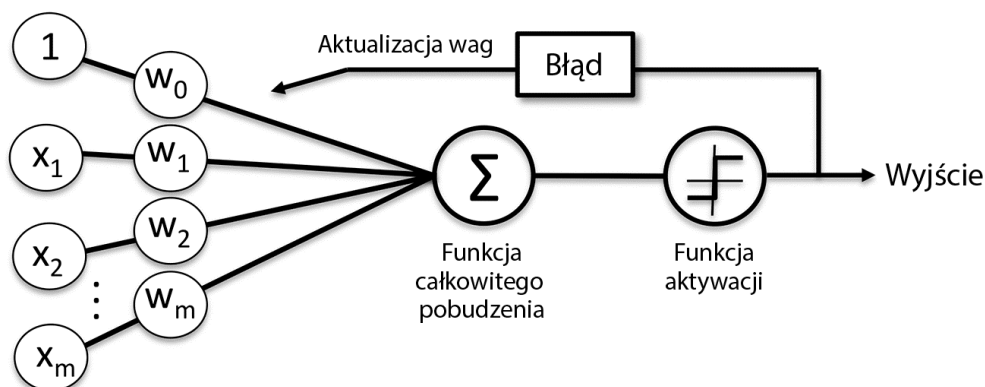


Rysunek 2.3. Rozdzielność liniowa klas

Kod źródłowy

Pliki kodu źródłowego są dostępne do pobrania pod adresem <ftp://ftp.helion.pl/przyklady/pythum.zip>.

Zanim przejdziemy do implementacji algorytmów w Pythonie, przyjrzyjmy się rysunkowi 2.4, stanowiącemu podsumowanie ogólnej koncepcji kryjącej się za modelem perceptronu.



Rysunek 2.4. Ogólny model perceptronu

Na rysunku 2.4 widzimy schemat ukazujący sposób, w jaki perceptron otrzymuje dane wejściowe próbek x i łączy z wagami w w celu obliczenia funkcji całkowitego pobudzenia. Wynik jest następnie przekazywany funkcji aktywacji (w tym przypadku funkcji skoku jednostkowego), która generuje wartość binarną -1 lub $+1$ — prognozowaną etykietę klas danej próbki.